

سكربت سكرين 2: Spark

بتمشي مع السلايدز بالترتيب · حوالي 56 دقيقة كلام
قبل النوتبوك

الكلام اللي هتقوله، بالترتيب **قول**

رسم بسيطة على البورد، وتحتها خطوات
رسمها **ارسم**

اسأل الطلبة، استنى يفكروا، وبعدين دوس
«الإجابة» **اسأل**

حاجة لازم تعرفها قبل ما توصل للسلايد **خد بالك**

DataFrames والكسل

Spark من جوه

ليه Spark؟

سلايد 1 – 6 · 8 د

ليه Spark؟

Lab 2 — Spark for Big Data

قول

الأسبوع اللي فات اللابتوب تعب لما كبرنا
الداتا. فاكرين؟

النهارده هنتعرف على الأداة اللي اتعملت
مخصوص عشان المشكلة دي: **Apache**
Spark.

اسأل

حد فاكّر الجهاز وقف عند كام شهر من داتا
التاكسي؟

- الإجابة

مش مهم الرقم بالضبط. المهم إن كان فيه
حد، والـ **RAM** اتملت. النهارده هنشوف إزاي
نعدّي الحد ده.

What is Big Data?

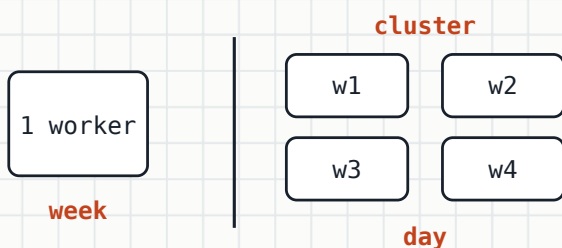
قول

Big Data يعني داتا كبيرة أو سريعة أو ملخبطة لدرجة إن جهاز واحد مش قادر يخلصها في وقت معقول.

شيت إكسل فيه 10 آلاف صف؟ عادي جدًا.
كل عمليات الفيزا في مصر في يوم واحد؟
مستحيل على جهاز واحد.

الحل: بدل جهاز واحد شايل الشغل كله،
نقسم الداتا والشغل على أجهزة كثير شغالة
مع بعض. الأجهزة دي اسمها **cluster**
(مجموعة أجهزة بتشتغل كأنها جهاز واحد).

ارسم على البورد



1. على الشمال: راجل واحد قدامه كومة ورق كبيرة، واكتب تحته «أسبوع».
2. على اليمين: 4 رجالة، كل واحد قدامه كومة صغيرة، واكتب تحتهم «يوم».
3. اكتب فوق اليمين كلمة **cluster**.

عندك 10 آلاف ورقة امتحان. مصحح واحد
ولا 100 مصحح؟ وليه؟

- الإجابة

100 مصحح طبقًا، كل واحد ياخذ 100 ورقة
ويخلصوا في يوم. وده بالظبط فكرة الـ
cluster: نقسّم الشغل ونشتغل على
التوازي (**parallel**).

The 3 Vs of Big Data

قول

Volume: الحجم. زي كل الصور اللي بتترفع على إنستجرام.

Velocity: السرعة. البنك لازم يعرف في ثواني إن عملية الكارت دي نصب ولا لأ.

Variety: التنوع. تويته غير جدول بنك غير فيديو.

في المحاضرة الدكتور قال 4، وهنا 3. العدد بيختلف من كتاب للتاني والفكرة واحدة.

اسأل

لازم التلاتة يتحققوا مع بعض عشان نقول دي **Big Data**؟

- الإجابة

لأ. كفاية واحدة بس تبقى مشكلة. مثلاً داتا مش كبيرة بس جاية بسرعة رهيبه، دي محتاجة أدوات **Big Data** برضه.

The Big Data Tools Landscape

قول

مفيش أداة واحدة بتعمل كل حاجة. كل أداة
ليها دور.

متحفظوش الأسامي. افكروا الأدوار: تخزين،
ونقل داتا، ومعالجة، واستعلام.

مثال واحد يكفي: **Kafka** زي البوسطجي،
بيوصل الداتا من مكان لمكان من غير ما
يقراها. و **Spark** هو اللي بيقرأ ويحلل.

Hadoop vs. Spark: Not Rivals

قول

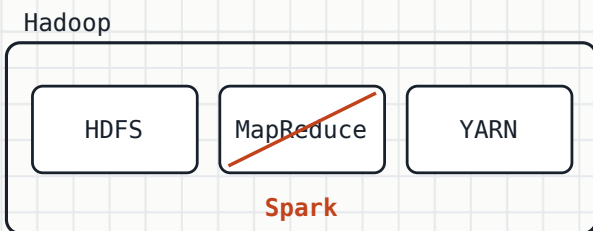
ناس كتير فاكدة إن **Hadoop** و **Spark** منافسين. لأ، الاتنين بيكملوا بعض.

Hadoop فيه 3 حاجات: **HDFS** (بيخزن الملفات على أجهزة كتير)، و **MapReduce** (بيعالج، بس بطيء لأنه بيكتب على الهارد بعد كل خطوة)، و **YARN** (بيوزع الموارد).

Spark بيعالج بس، ومعدوش مكان يخرن فيه. فيقرا من **HDFS** أو من الكلاود.

يعني **Spark** بيشيل **MapReduce** ويقعد مكانه. مش بيشيل **Hadoop** كله.

ارسم على البورد



1. ارسم مربع كبير واكتب فوقه **Hadoop**.
2. جواه 3 مربعات: **HDFS**، و **MapReduce**، و **YARN**.
3. اشطب على **MapReduce** واكتب جنبه **Spark** بلون تاني.

لو هشتغل بـ **Spark**، لازم يبقى عندي **Hadoop**؟

- الإجابة

لأ. **Spark** ممكن يشتغل لوحده. وإحنا النهارده هنشغله على **Colab** من غير **Hadoop** خالص.

Why Apache Spark in This Lab?

قول

ليه **Spark** بالذات؟ أهم سببين:

سريع، لأنه بيشتغل **in-memory** (بيحط الداتا في ال **RAM** بدل الهارد).

الكود اللي هتكتبه على اللابتوب هو هو اللي هيشغل على **cluster** فيه 100 جهاز.

وينكتبه **Python** عن طريق **PySpark**،
وبيشتغل على **Colab** بسطر واحد.

اسأل

ال **RAM** ولا الهارد أسرع؟ وده يفرق في إيه؟

- الإجابة

ال **RAM** أسرع بكثير جدًا. عشان كده **Spark** أسرع من **MapReduce**: بيخلي النتائج في الذاكرة بدل ما يكتبها على الهارد كل خطوة.

نص دقيقة

عدّي بسرعة

سلايد 7

Agenda

قول

ده اللي جاي: **Spark** إيه، ومبني إزاي، والـ **DataFrames**، وبعدين نكتب كود.

1.5 د

سلايد 8

What is Apache Spark?

قول

Spark هو **engine** (محرك) بيعمل الحسابات على داتا متوزعة على أجهزة كثير.

بتكلّمه بـ **Python** أو **Java** أو **Scala** أو **R**.

وفوقه مكتبات جاهزة: **Spark SQL** للجداول، و **MLlib** للـ **machine learning**، و **Structured Streaming** للداتا اللايف، و **GraphX** للعلاقات.

بصوا على الرسمه: كل المكتبات قاعدة فوق نفس الـ **engine**.

Spark's Philosophy

قول

فكّروا فيه زي مطعم كبير.

فيه شيف واحد شاطر جدّا: ده ال **engine**.

وعنده أقسام: مشويات وحلويات وسلطات.
دي المكتبات.

والشيف مش يزرع الخضار بنفسه، بيحييه
من المورّد. وده معناه إن **Spark** مش
بيخزّن، بيقرأ الداتا من مكان تاني.

Spark's Architecture

قول

دي أهم سلايد في الجزء ده. هرسمها على البورد.

Driver: المخ. ييقرا الكود بتاعك ويقسّمه لشغلانات صغيرة.

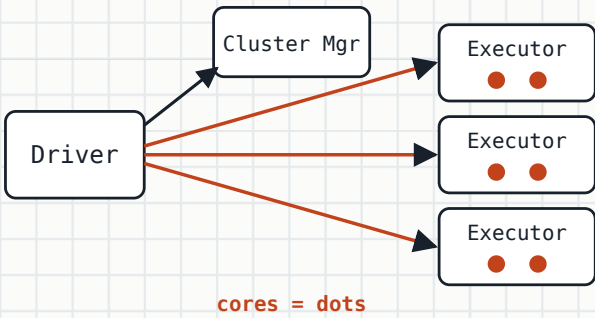
Cluster Manager: زي ال **HR**، بيوفّر أجهزة وموارد.

Executors: الموظفين اللي بيشتغلوا بجد. كل واحد على جهاز.

Cores: إيدين الموظف. كل **core** بيعمل **task** (مهمة صغيرة) في نفس الوقت.

الترتيب: ال **Driver** يطلب من ال **Cluster Manager** أجهزة، وبعدين يكلم ال **Executors** على طول ويوزّع عليهم الشغل.

ارسم على البورد



1. مربع على الشمال واكتب فيه **Driver**.
2. في النص مربع صغير واكتب فيه **Cluster Manager**.
3. على اليمين 3 مربعات **Executor**، وجوا كل واحد دائرتين صغيرين (**cores**).
4. سهم من ال **Driver** لل **Manager**، وأسهم من ال **Driver** لل **Executors** على طول.

اسأل

مين فيهم شایل الداتا؟ ال **Driver** ولا ال **Executors**؟

- الإجابة

ال **Executors**. ال **Driver** بيخطط ويوّرّع بس، زي المدير اللي مش بيشيل الكراتين بنفسه.

لو عندنا 3 **executors** وكل واحد فيه 4 **cores**، يقدرُوا يعملُوا كام **task** في نفس اللحظة؟

- الإجابة

$$4 \times 3 = 12$$

Spark Applications

قول

نفس الكلام بمثال بفلوس.

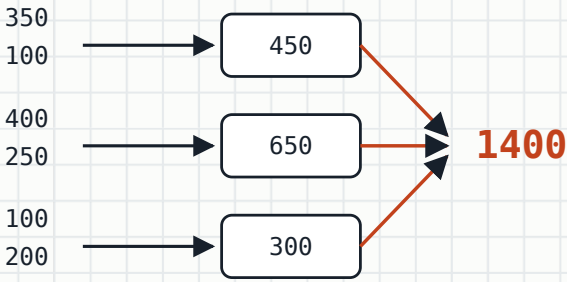
المدير (**Master**) معاه جدول فيه 6 مبالغ.
بيقسّمه على 3 عمال (**Workers**)، كل واحد
ياخد صفين.

كل عامل يجمع اللي معاه لوحده، والمدير
يجمع النتائج الصغيرة، فيطلع الإجمالي
1400.

كل حته من الجدول دي اسمها **partition**
(جزء من الداتا). وده سر السرعة: كل
الأجزاء بتشتغل في نفس الوقت.

و **SparkSession** هي الباب اللي بندخل
منه لـ **Spark** في الكود.

ارسم على البورد



1. اكتب 6 أرقام عمودي على الشمال.
2. قسّمهم 3 مجموعات، كل مجموعة رقمين، وارسم سهم لكل مجموعة.
3. اكتب مجموع كل مجموعة، وبعدين سهم للإجمالي على اليمين.

اسأل

لو عامل منهم اتأخر، الإجمالي هيطلع إمتى؟

- الإجابة

مش هيطلع غير لما أبطأ عامل يخلص.
وعشان كده تقسيم الداتا بالتساوي على ال-
partitions مهم.

قول

DataFrame يعني جدول: صفوف وأعمدة.
زي اللي شغناه في **Pandas** الأسبوع اللي فات.

الفرق إن الجدول هنا ممكن يكون متقطّع
على أجهزة كثير.

الكود ده بيعمل جدول فيه عمود واحد اسمه
number وفيه الأرقام من 0 لـ 999.

و `show()` بتعرض أول 20 صف بس،
عشان الشاشة متتملاش.

اسأل

الجدول ده فيه كام صف بالضبط؟

- الإجابة

1000 صف، من 0 لحد 999. إحنا شايفين 20
بس لأن `show()` بتعرض 20 بشكل
افتراضي.

Transformations

قول

Transformation (تحويل) يعني عملية بتطلّع جدول جديد. الجدول الأصلي عمره ما يتغير، وده اسمه **immutability**.

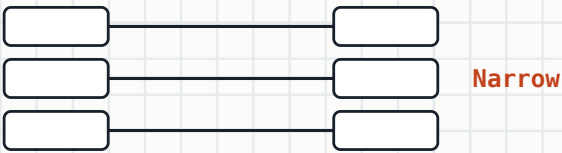
أمثلة: filter (هات الصفوف اللي فيها شرط معيّن) و select (هات أعمدة معيّنة).

فيه نوعين:

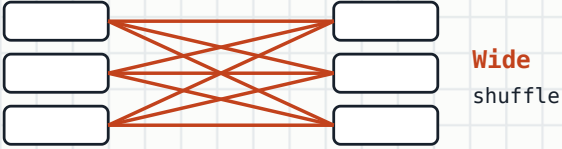
Narrow: كل جزء بيشتغل لوحده في مكانه. سريع.

Wide: الأجزاء محتاجة تبدّل داتا مع بعض على الشبكة. النقل ده اسمه **shuffle**. وده بطيء.

ارسم على البورد



Narrow



Wide
shuffle

1. عمودين مربعات (شمال ويمين)، كل عمود فيه 3 مربعات.

2. فوق: أسهم مستقيمة، كل مربع لمربعه. اكتب

Narrow

3. تحت: أسهم متقاطعة من كل مربع لكل

المربعات. اكتب Wide = shuffle

اسأل

الفصل كله عايز يرتب نفسه حسب الطول.
ده **Narrow** ولا **Wide**؟ وطيب لو كل واحد
بس يعدّ الكراسي اللي في صفه؟

- الإجابة

الترتيب حسب الطول **Wide**، لأن الناس
لازم تقوم وتبدّل أماكنها. وعدّ الكراسي
Narrow، كل واحد في مكانه.

filter على المرتب ولا **groupBy**

حسب القسم، أنهى فيهم محتاج **shuffle**؟

- الإجابة

groupBy ، لأن كل موظفين القسم الواحد
لازم يتجمعوا في مكان واحد. أما filter
فكل جزء يفلتر نفسه.

Lazy Evaluation & Actions

قول

دي أهم فكرة في السكشن كله.

Spark كسلان (**lazy**). لما تكتب **transformation** مش بينفذها، بيكتبها في خطة (**plan**) وخلاص.

التنفيذ بيحصل بس لما تطلب نتيجة بجد. ده اسمه **Action**، زي **count** و **take** و **collect**.

ليه؟ عشان لما يشوف الخطة كلها مرة واحدة، يرتّبها أذكى. مثلاً يفلتر الأول وبعدين يرتّب، فيرتّب داتا أقل.

ارسم على البورد

bread	T
milk	T
eggs	T

plan only

go shopping	A
-------------	---

runs here

1. اكتب ورقة طلبات سوبر ماركت: عيش، لبن، بيض. وجنب كل واحدة اكتب **T**.
2. تحت خالص اكتب «انزل اشترى» واكتب جنبها **A**.
3. ارسم سهم من **A** لفوق: «هنا بس التنفيذ يحصل».

اسأل

إنت بتنزل السوبر ماركت كل ما تفتكر حاجة، ولا بتستنى الليسته تخلص؟

- الإجابة

بتستنى الليسته تخلص وتنزل مرة واحدة. ده بالظبط ال **lazy evaluation**: ال **transformations** هي الليسته، وال **action** هو إنك تنزل.

كتبت 5 **transformations** ورا بعض ومفيش **Spark.action** اشتغل قد إيه؟

- الإجابة

ولا ثانية. مفيش أي حاجة اتنفذت لسه، كله مكتوب في الخطة بس.

End-to-End Example: Flight Data

قول

داتا حقيقية: رحلات طيران سنة 2015. كل صف فيه: رايحة فين، وجاية مين، وكام رحلة.

header=True يعني أول سطر في الملف هو أسامي الأعمدة.

inferSchema=True يعني Spark يخمن نوع كل عمود. من غيرها كله هيتقري نص، حتى الأرقام.

و take(3) action، فيجيب أول 3 صفوف.

اسأل

لو شلنا inferSchema ، وجينا نرّتب حسب عدد الرحلات، إيه اللي هيحصل؟

- الإجابة

هيرتبهم كنص مش كأرقام، فال 100 هتيجي قبل ال 9، لأن الحرف 1 قبل الحرف 9.

Sort, explain & SQL functions

قول

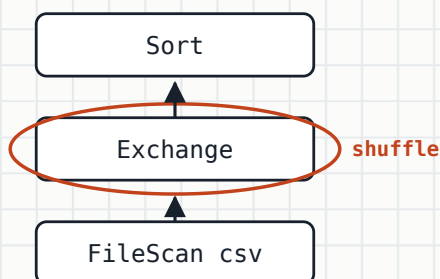
sort **transformation**، يعني بيطلّع
جدول جديد ومش بيلمس القديم.

explain بتوّرّيك الخطة اللي **Spark** ناوي
ينفذها، من غير ما ينفذها.

الخطة بتتقري من تحت لفوق: يقرأ الملف،
بعدين **Exchange**، بعدين يرّتب.

و max بتجيب أكبر عدد رحلات: 370002.

ارسم على البورد



1. 3 مربعات فوق بعض، والسهم طالع لفوق.
2. تحت: **FileScan** (قراءة الملف). في النص:
Exchange. فوق: **Sort**.
3. لف دائرة على **Exchange** واكتب جنبها:
shuffle.

اسأل

ليه في الخطة ظهرت كلمة **Exchange**؟
إحنا ما طلبناش نقل داتا.

- الإجابة

لأن الترتيب عملية **wide**: عشان ترتّب كل
الداتا لازم تنتقل بين الأجزاء. يعني
Exchange هي ال **shuffle** اللي اتكلمنا
عنه.

باقي الأدوات و RDDs

سلايد 17 - 22 · 7 د

دقيقة

سلايد 17

A Tour of Spark's Toolset

قول

Spark مش جداول بس.

Structured Streaming: نفس الكود
تشغله على داتا لايف بتعديل بسيط.

MLlib: machine learning على داتا
ضخمة: **classification** (توقع فئة)، و
regression (توقع رقم)، و **clustering**
(تجميع المتشابه).

هنرجع لهم في سكاشن جاية.

Structured Streaming: Flight Data

قول

قراءة عادية: `spark.read` . قراءة لايف:
`spark.readStream` .

في اللايف مينفعش `inferSchema` . لازم
تدّيله ال `schema` (أسامي الأعمدة وأنواعها)
بنفسك.

الحل: نقرا الملف مرة قراءة عادية، وناخذ
منه ال `schema`، ونديها لل `stream`.

الجزء ده مش في النوتبوك، نظري بس.

اسأل

ليه `Spark` مش عارف يخمّن الأنواع في
الداتا اللايف؟

- الإجابة

لأن الداتا لسه بتوصل، فمفيش داتا كاملة
قدامه يبص عليها ويخمّن منها.

دقيقة

عدّي بسرعة

سلايد 19

Datasets

قول

Dataset نسخة من الـ **DataFrame** بتكتشف الأخطاء وإنت بتكتب الكود، قبل ما يشتغل.

بس خدوا بالكم: موجودة في **Java** و **Scala** بس. في **Python** مش موجودة.

إحنا هنشتغل بـ **DataFrames** و **RDDs**، فدي معلومة للعلم.

خد بالك

السنين اللي في رسمة الـ **timeline** مش مطبوعة. لو حد سأل: **Spark 1.0** سنة 2014، و **1.3** سنة 2015، و **1.6** سنة 2016. والمهم الترتيب: **RDD** ثم **DataFrame** ثم **Dataset**.

نص دقيقة

عدّي بسرعة

سلايد 20

Creating a Dataset

قول

ده كود **Scala**، ومش هنكتبه. عدّي بسرعة.

Low-level APIs: RDDs

قول

RDD هو الأساس اللي **Spark** كله مبني عليه.

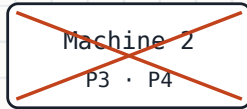
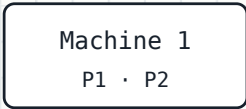
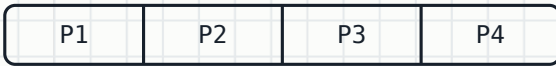
Resilient: لو جهاز وقع، **Spark** يعيد حساب الجزء اللي ضاع، لأنه فاكّر الخطوات.

Distributed: متقسّم على أجهزة.

Dataset: أي مجموعة عناصر، أرقام أو نص أو أي حاجة.

بنستخدمه لما الداتا مش جدول، زي نص أو صور، أو لما عايزين نتحكم في كل حاجة بإيدنا.

ارسم على البورد



recompute P3, P4

1. مستطيل طويل واكتب فيه **RDD**.
2. قطّعه 4 حتت (**P1** لحد **P4**).
3. ارسم جهازين، وكل جهاز ياخذ حتتين.
4. اشطب على جهاز واحد، واكتب: «**Spark** يعيد حساب حتتينه».

اسأل

لو الجهاز الثاني وقع، الداتا اللي عليه ضاقت؟

- الإجابة

لأ. **Spark** فاكر الخطوات اللي طلّعت الحتت دي، فبيعيد حسابها على جهاز ثاني. وده معنى **Resilient**.

SparkSession vs. SparkContext

قول

زمان كان فيه 3 أبواب: **SparkContext**، و **SQLContext**، و **HiveContext**.

دلوقتي باب واحد: **SparkSession**.

ولو محتاج الباب القديم عشان الـ **RDDs**،
خده منها: `spark.sparkContext`.

اسأل

ينفع أعمل **SparkContext** جديد وأنا عندي
SparkSession شغالة؟

- الإجابة

لا، هيطلع **error**، لأن **Spark** بيسمح بواحد
بس في نفس الوقت. وهنشوف ده في
النوتبوك.

Lab: Colab Setup & Data

قول

`files.upload()` بتفتح نافذة ترفع منها
ملف من جهازك على **Colab**.

`os.listdir` بتعرض الملفات اللي
اترفعت، عشان تتأكد إنها موجودة.

خد بالك

السلايد بتقول ارفع **data.zip** وفكّه، بس الدكتور ما
بعتش **zip**. الطلبة هيرفعوا الـ **CSV** على طول، زي
ما النوتبوك بيعمل. قولها لهم صريحة عشان
ميدوروش على ملف مش موجود.

Lab: Setup & first DataFrame

قول

نفس اللي شرحناه في سلايد 15، بس المرة دي هتكتبوه بإيديكم.

اعملوا **SparkSession**، واقرأوا الملف،
وبعدين `take(5)` و `explain`.

اسأل

قبل ما نشغل `take(5)` : مين فيهم **action**
ومين **transformation**؟ `read` و `sort` و
`take` و `explain`.

- الإجابة

`take` **action** أكيد. و `sort`
transformation. و `explain` مش
action ومش **transformation**، دي بس
بتعرض الخطأ.

خد بالك

المسار المكتوب في السلايد
(`/content/data/data/...`) غلط بالنسبة لملفات
الدكتور. المسار الصح: `-/content/2015-`
`summary.csv`

Lab: Create an RDD from a collection

قول

spark.sparkContext يجيب الباب
القديم، لأننا محتاجينه للـ **RDDs**.

list(range(10)) ليستة عادية في
Python فيها من 0 لـ 9.

parallelize بيقطّع الليستة **partitions**
ويوزّعها.

getNumPartitions بتقول اتقسمت كام
حثة. عند الدكتور طلعت 2.

اسأل

ليه طلعت 2 **partitions** بالذات؟

- الإجابة

لأن جهاز **Colab** المجاني فيه 2 **cores**، فـ
Spark قسّم على قدهم.

خد بالك

فيه خلية في النوتبوك فيها `SparkContext()` . دي
هتطلّع **error** لأن فيه **session** شغالة أصلاً. قول
للطالبة يعدّوها.

RDDs are Immutable

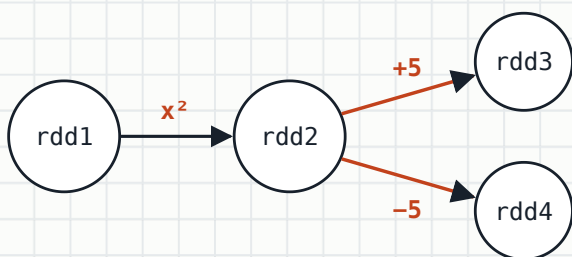
قول

map بتطبق دالة على كل عنصر.

rdd3 طلغ rdd2 مربعات rdd1. ومن rdd2 طلغ rdd3
(زائد 5) و rdd4 (ناقص 5).

rdd1 و rdd2 ما اتغيروش خالص. كل خطوة
بتعمل واحد جديد.

ارسم على البورد



1. دائرة rdd1، وسهم لدائرة rdd2 واكتب على
السهم x^2 .

2. من rdd2 سهمين: واحد لـ rdd3 (+5) وواحد لـ
rdd4 (-5).

اسأل

rdd1 فيه إيه دلوقتي بعد كل التحويلات دي؟

- الإجابة

زي ما هو بالظبط: من 0 لـ 9. ده الـ

immutability.

Lazy Evaluation in Action

قول

دي أحلى سلايد في السكشن. هنتبت إن
Spark كسلان.

اكتبوا قسمة على صفر. شغلوا. مفيش
!error

دلوقتي اكتبوا `take(2)` . هوب، طلع
.ZeroDivisionError

اسأل

ليه ال **error** ما ظهرش غير مع `take` ؟

- الإجابة

لأن `map transformation`، ف **Spark** كتبها
في الخطة ومنفذهاش، فما اكتشفش
المشكلة. أما `take` ف **action**، يعني نَقِّذ
فعلاً، وساعتها لقي القسمة على صفر.

خد بالك

قبل ما تشغل السطر الثاني، خليهم يتوقعوا الأول:
«هيطلع **error** ولا لأ؟» وبعدين شغل. ورسالة ال
error طويلة جدًا، فقولهم يبصوا على آخر سطر
بس.

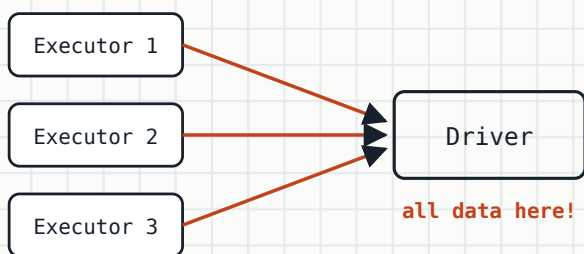
collect(): an Action

قول

collect يجيب كل العناصر من كل ال
executors ويحطهم عند ال **Driver**.

persist بتقول لـ **Spark**: احتفظ بالنتيجة
دي في الذاكرة، عشان لو احتجتها تاني
متحسبهاش من الأول.

ارسم على البورد



1. 3 مربعات **Executor** على الشمال.
2. أسهم كلها رايحة لمربع واحد على اليمين:
Driver.
3. اكتب جنب ال **Driver**: «لو الداتا ضخمة، ال
Driver هيتخنق».

اسأل

عندك جدول فيه مليار صف. تعمل
collect عادي؟

- الإجابة

لأ! ال **Driver** جهاز واحد، وذاكرته هتتملي
والبرنامج هيوقع. استخدم `take(n)` عشان

1.5 د

سلايد 29

Lambda Functions

قول

lambda طريقة مختصرة نكتب بيها **function** في سطر واحد.

```
def sq(x): return x**2  
lambda x: x**2
```

بنستخدمها جوه `map` عشان منقعدش نعرّف **function** كاملة لحاجة صغيرة.

اسأل

اكتبلي **lambda** بتضرب أي رقم في 10.

- الإجابة

```
lambda x: x * 10
```

Lab: RDD from a text file

قول

textFile بتقرا ملف نصي، وكل سطر
بيبقى عنصر.

split بتقطع السطر كلمات.

map بتطلع ليستة لكل سطر، يعني ليستة
جوه ليستة.

flatMap بتفرد كل الكلمات في ليستة
واحدة.

ارسم على البورد

"second line" . "the third line"

map

flatMap

[second, line]
[the, third, line]

second, line, the,
third, line

1. اكتب سطرين: «second line» و «the third line».

2. تحت الشمال (map): [second, line], [the, third, line].

3. تحت اليمين (flatMap): [second, line, the, third, line].

اسأل

لو عايز أعدد كل كلمة اتكررت كام مرة في الملف، أستخدم أنهي واحدة؟

- الإجابة

flatMap، لأنني محتاج كل الكلمات في ليستة واحدة عشان أعدّها. ودي أول خطوة في مثال **Word Count** المشهور، اللي هو أساس **MapReduce**.

خد بالك

الكلام اللي في السلايد (**hello spark**) مثال بس. ملف الدكتور فيه 4 سطور تانية، فالناتج هيطلع مختلف. قولها للطلبة قبل ما يتلخبطوا.

خلصت السلايدز؟ افتح النوتبوك وابدأ من

```
pip install pyspark
```